PUP-SPOT Group 5 progress - 29 October 2026

Team and application

PUP-SPOT (System for Property Ownership Tracker), a campus lost-and-found system using native PHP, PDO and PostgreSQL. Members: Raguin, Justine Philip; Cueto, Daniel Luigi; Imperial, Aaron Paul; Manansala, Emmanuel John; Napeñas, Jenver. The original submitted proposal is preserved.

Repository: https://github.com/sanatorion/grp05_lostandfound (private; other testing groups need access from the owner).

Evidence-backed milestones

Date	Work	Evidence / state
Oct 26	Reviewed course activities and original Group 5 proposal/SQL; registered team and title	Local course-review source copies and tracker Directory row 7
Oct 29	Cloned main at 5fcbccd, configured PHP 8.5.10 and PostgreSQL 18.3, initialized five tables and sample records	Local setup verification; app uses restricted pupspot_runtime account
Oct 29	Corrected setup documentation and added placeholder environment template / read-only Postman collection	Commit 612dd68, PR #22; awaiting review
Oct 29	Implemented five ready administrative retrieval endpoints	feature/admin-record-retrieval; issues #1, #4, #7, #11, #13; awaiting review
Oct 29	API validation	New collection: 32 requests, 61 assertions, zero failures. Existing seven requests / 14 assertions pass. All 15 PHP files pass syntax checks. Empty-list test rolls back its changes.
Oct 29	Local DBA rehearsal	Backup, restore, integrity comparisons, allowed SELECT/UPDATE, denied CREATE TABLE, execution plan, VACUUM ANALYZE and recovery into separate disposable databases; source unchanged

Working behavior and verification

See administrative retrieval for routes, response codes and test instructions. Responses use actual stored fields. Administrative reports and claims are not exposed in the public namespace. New read endpoints do not introduce application logins, change schema, or implement matching/notifications.

See user roles for intended student/staff/DBA mapping and actual local privileges. The current shared application account is not a completed public/admin database-role separation.

The local backup is 19,346 bytes, custom pg_dump format, with an archive listing and SHA-256 recorded in the DBA rehearsal evidence. All five restored table row counts and row-content hashes matched the source. Constraint inventory matched and invalid statuses were rejected. A deletion on the disposable restored copy was recovered from the snapshot into another fresh database. The source database was unchanged. Backup files and credentials remain outside Git.

The first restore comparison failed because PostgreSQL rendered equivalent CHECK array casts differently after restore. Verification was corrected to compare constraint metadata and test rejection of invalid values, alongside exact row hashes. No database constraint or original schema was changed to force a pass. Elapsed backup/restore timings were not retained, so no recovery-time performance claim is made.

Remaining work

- Project lead reviews/integrates pull requests. Ready board issues remain open until accepted; implementation is not merged completion.

- Backlog write endpoints, public browsing/search, reference-code cancellation, fuzzy matching, notification delivery and frontend remain unfinished. Lead-owned matching/report-submission/notification work was not duplicated.

- Category POST/PATCH handlers are absent from current main despite Done cards. Query-string routing remains a known gap. These findings require integration follow-up.

- TextBee and email need actual service configuration and live validation. No message was sent; prior TextBee success-path checks used a local stub.

- Reconcile documented proposal/schema differences with the project lead without editing the original submission. Review the seed script before rerunning it: it contains destructive TRUNCATE behavior and is intended for disposable initialization.

- Independently install and test the complete app on another machine. The class tracker currently lists no other groups' repository links; cross-group Activities 3-6 have not been executed.

- Midterm pairing, scenario approval/release and defense schedule require instructor/partner information. No unassigned incidents or answers were entered into the tracker.

Backup plan for review

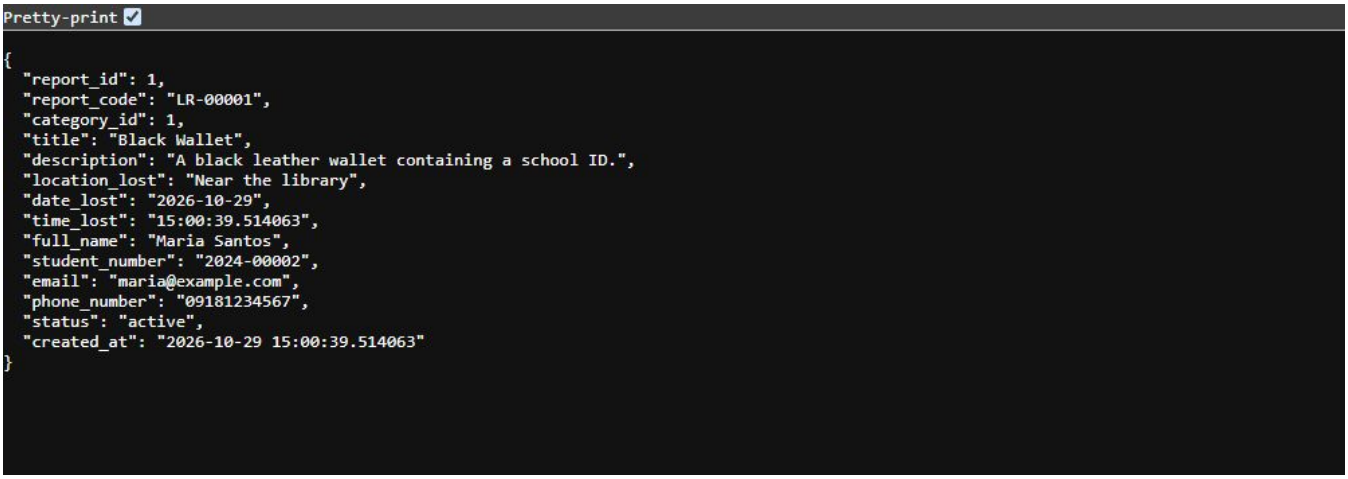
Proposed development policy: daily backup and before significant database changes; keep seven daily backups outside the repository; target RPO 24 hours and RTO 30 minutes for the small course database. This is a proposed policy, not an installed scheduler or achieved recovery SLA. Restore to a fresh database, recreate cluster roles and reapply required grants, then verify data, constraints and application behavior before switching connections. Single-database dumps omit cluster-wide roles. Snapshot restore loses changes made after the snapshot; point-in-time recovery is not configured.

AI usage declaration

OpenAI Codex assisted with course review, environment setup, PHP retrieval code, test collections, documentation and tracker entry preparation. Validation consisted of actual local execution, PHP syntax checks, Postman/Newman requests, transaction rollback checks and isolated PostgreSQL backup/restore exercises. AI assistance does not establish instructor approval or teammate authorship. All five members should review the code and evidence and be able to explain them before submission/defense. This report records progress and is not a declaration that every course activity is complete.

Working endpoint evidence

GET http://127.0.0.1:8000/api/admin/lost-reports/1 - captured after the local implementation. The displayed record is supplied fictional seed data.



The full application UI and create/update workflows remain pending; this screenshot demonstrates retrieval only.